实验 6：工具调用、Agent 与 Harness 实现

姓名：罗心月　　学号：24111302025　　班级：一班

1. 实验目的

- 掌握工具接口定义、参数验证、权限边界和结果反馈。
- 实现一个由模型根据任务和工具结果决定下一步操作、运行状态可查看且能够终止的单 Agent。
- 编写运行时 Harness，完成上下文组织、工具执行、结果检查、停止控制和日志记录。

2. 实验内容

2.1 任务选择与边界

- 结合个人软件选择一个范围较小、结果能够检查的任务，如查询样例数据后生成统计结果；不适合接入个人软件时，可独立完成练习。
- 列出任务目标、可用数据、工具、停止条件和成功标准，并给出一组输入及预期结果。本实验实现单 Agent，由模型根据目标和工具结果决定下一步操作；说明哪些步骤由程序预设，哪些由模型决定。
- 说明哪些操作允许自动执行，哪些必须用户确认，哪些明确禁止。

2.2 工具接口与参数约束

- 实现至少 2 个可实际运行的工具，如读取样例文件、查询数据或执行统计计算，说明名称、用途、输入字段、类型、必填项、返回结果和错误信息。输入格式及字段约束也称 Schema。
- 建立允许调用的工具列表，执行前由程序检查工具名称、参数和访问范围，例如数值范围、允许的文件目录或可访问的数据；不能直接执行模型生成的任意函数名或命令。
- 说明工具的最长等待时间、是否修改数据，以及重复调用是否会产生重复修改；在接入 Agent 前，直接测试至少一次正确参数和一次错误参数。

2.3 Agent 循环与状态

- 绘制“接收目标—模型提出调用请求—程序检查并执行工具—回传工具结果—模型继续或给出最终回答”的循环图，标出参数校验、人工确认和停止检查的位置。
- 接入支持工具调用的模型接口，或约定可由程序解析的 JSON 调用格式，实际运行模型与工具之间的循环。模型提出工具名称和参数，程序负责执行；将调用请求和对应工具结果按接口要求加入后续上下文，不能只把结果打印出来。
- 设置最大执行步数、模型与工具调用超时、总运行时限及重试上限，说明一步如何计数，并由程序执行限制。模型给出最终回答后，还应检查结果是否满足 2.1 的成功标准，不能仅凭模型声称完成就判定成功。

2.4 运行时 Harness 设计与实现

- 将组织模型与工具运行的外围程序作为本实验的运行时 Harness，说明任务输入、上下文、执行、结果检查和日志分别由哪些函数或模块负责，不要求拆成多个独立服务。
- 实现模型调用、工具分发、参数与权限检查、结果回传和停止控制；保留当前任务需要的输入、调用记录及工具结果。关键限制必须由程序执行，不能只写在提示词中。
- 工具失败或结果不符合要求时，将错误信息反馈给模型进行有限次修正，或结束任务并说明原因。可以使用开发框架或自行编写代码，但须说明实际控制流程，不要求多 Agent 或复杂框架。

2.5 失败处理与可观测性

- 为每次运行设置任务编号，记录时间、调用的工具与参数摘要、工具结果、错误类型、执行步数、耗时和最终状态；密钥和个人信息须遮挡或省略。
- 处理参数错误、工具异常、超时和达到步数上限等情况，返回明确状态和原因。超时可能只是停止等待，不代表后台工具已停止，应说明实际取消能力；可能修改数据的调用状态不明时，先核对执行结果，不得直接重试。
- 对需要确认的操作，在执行前展示操作内容与影响，由用户明确同意后执行；拒绝或等待超时则不执行。

2.6 测试与结果分析

- 至少测试 5 类情况：正常完成、参数错误、工具异常、达到步数或时间限制、人工确认。其中人工确认分别检查同意与拒绝；异常与边界情况可模拟触发，正常流程必须实际调用模型和工具。
- 用表格记录输入、预期行为、实际结果和判断依据，检查任务是否完成、错误是否正确反馈、停止与确认规则是否生效；未符合预期时，分析问题出在模型决策、参数、工具、检查规则还是控制流程。
- 说明 Agent 及运行时 Harness 是否用于个人软件，以及当前局限和改进计划；在个人 GitHub/Gitee 仓库保存工具、Agent、Harness、测试代码和运行说明，报告注明路径及对应 Commit。

3. 操作记录

以下截图、代码和日志均编号并配文字说明。数据来自 2026 年 11 月 9 日在本机的实际运行；正常流程由程序真实调用 DeepSeek 的 chat completions 接口并执行工具，异常、边界和确认场景使用脚本模型模拟模型的决策，工具与 Harness 仍为真实实现。

3.1 任务选择与边界

任务接在个人软件 StudyPilot 上：用户给出“统计课程最近 14 天任务完成情况并给建议”的目标，Agent 通过工具查询项目自己的 SQLite 样例数据，算出完成率、各知识点完成情况和掌握度最低的知识点；若完成率低于 60%，再为最薄弱知识点创建一条 30 分钟复习任务，写入前必须由用户确认。任务范围小，统计数字和是否写入都可以由程序直接核对，适合作为本实验的单 Agent 任务。对应实验 2 的用户故事是“作为本科生，我希望把课程要求转换成可修改的学习计划，以便按时完成任务并了解自己的薄弱知识点”，本实验补充的是计划执行后的统计与调整环节。

表 6-1: 任务目标、数据、工具、停止条件与成功标准

项目	内容
任务目标	统计课程“软件工程”最近 14 天的任务完成情况，找出掌握度最低的 2 个知识点并给出下一步建议；完成率低于 60% 时为最薄弱知识点创建 30 分钟复习任务（需确认）
可用数据	backend/data/studypilot.db 中的课程、考试目标、知识点、学习计划、每日任务表；样例场景为 12 条任务（5 条已完成、7 条未完成、合计 420 分钟）、4 个知识点（掌握度 62/45/38/30）、考试目标 85 分、统计窗口 2026-10-27 至 2026-11-09
可用工具	list_courses、query_tasks、task_stats、create_review_task
停止条件	模型给出最终回答且通过程序核验；或达到最大步数、总运行时限、模型调用失败、核验失败上限；人工确认被拒绝时按“未创建任务”收尾
成功标准	completion_rate=41.7、total_tasks=12、done_tasks=5、pending_tasks=7、薄弱知识点为“软件测试/数据库设计”、建议非空；若声称创建任务，数据库 daily_tasks 中必须确有该任务
输入示例	python3 -m app.tools.agent_cli --confirm ask，目标为“统计课程“软件工程”最近 14 天的任务完成情况……需要我确认”
预期结果	前 3 步依次调用 list_courses、task_stats、create_review_task；确认同意后返回 success，review_task_created=true，数据库中新增一条状态为 todo 的“复习：软件测试”任务；若拒绝则 review_task_created=false，数据库不新增任务

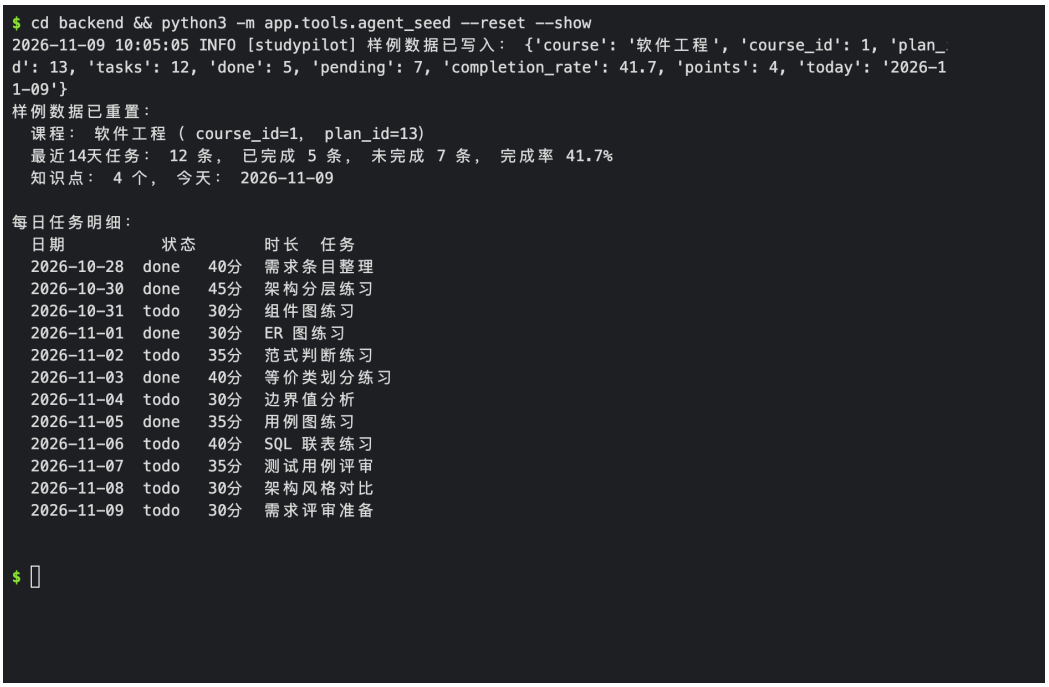


图 6-1: 样例数据：重置后写入 12 条任务，完成率 41.7%，另有知识点和考试目标

程序预设与模型决定的分工见表 6-2。模型只决定“下一步做什么”，所有会改变数据或影响安全边界的动作都由程序检查后执行。

表 6-2: 程序预设与模型决定的步骤

步骤	执行者
目标任务、课程、天数、步数/时限等参数	程序（命令行参数）
选择调用哪个工具、工具参数、何时给出最终回答	模型
工具名称白名单、参数 Schema 与取值范围、数据库访问范围	程序（执行前检查）
写操作展示内容与影响、等待用户确认	程序 + 用户
工具执行、单次超时、只读重试、总时限与步数检查	程序
工具结果与调用请求加入后续上下文	程序
最终回答的数字核对、写操作核对、成功/失败判定	程序
运行编号、调用记录、错误类型、耗时、最终状态	程序（JSONL 日志）

权限边界见表 6-3。前三项只读工具可以自动执行；写操作必须确认；模型不能直接发 SQL、表名、文件路径或命令。

表 6-3: 允许自动执行、必须确认和禁止的操作

类别	操作
允许自动执行	`list_courses`、`query_tasks`、`task_stats` 三个只读查询；模型调用；结果核验；日志写入
必须用户确认	`create_review_task`（向 `daily_tasks` 插入待办任务）；执行前显示工具、参数和影响，输入 y 同意，其他输入或 20 秒无输入视为拒绝
明确禁止	任意 SQL、表名、列名、文件路径或 shell 命令；白名单之外的函数名；修改或删除课程、任务、知识点；访问其他用户的数据；超出参数范围的调用（如查询 90 天、创建 99 分钟任务、日期早于今天）

3.2 工具接口与参数约束

共实现 4 个可运行工具，参数模型用 Pydantic 定义，既用于校验，也生成给模型的 JSON Schema。4 个工具都只访问当前用户的数据；SQL 在本项目中预先写死并使用参数绑定，模型无法传入 SQL 片段。接口说明见表 6-4。

表 6-4: 工具接口、参数 Schema、返回与错误信息

工具及用途	输入（字段: 类型，必填/约束）	返回结果与主要错误
`list_courses` 列出课程与考试目标（只读）	无参数	返回 `courses[]`（课程编号、名称、学期、考试日期、目标分）与 `count`；错误 `DATABASE_UNAVAILABLE`、`TOOL_TIMEOUT`
`query_tasks` 查询最近 N 天任务明细（只读，最多 50 条）	`course:str` 必填，1—30 字；`days:int` 可选，默认 14，1—30；`status:enum` 可选 all/todo/done	返回统计窗口、状态过滤、`tasks[]`（编号、标题、日期、分钟、状态、知识点、完成时间）、`summary`、`truncated`；错误 `INVALID_ARGUMENTS`、`COURSE_NOT_FOUND`、`DATABASE_UNAVAILABLE`、`TOOL_TIMEOUT`
`task_stats` 统计完成率、时长与薄弱点（只读）	`course:str` 必填，1—30 字；`days:int` 可选，默认 14，1—30	返回任务总数/完成数/未完成数、完成率（保留 1 位）、总/已完成/未完成分钟、逾期未完成数、`by_point[]`、`weakest_points[]`、`busiest_day`；错误同 `query_tasks`
`create_review_task` 创建待办复习任务（写操作，必须确认）	`course:str` 必填；`point:str` 必填且属于该课程；`minutes:int` 可选，默认 30，10—60；`date:str` 可选 YYYY-MM-DD，默认明天	返回 `created`、`task_id`、`title`、`date`、`minutes`；重复调用返回 `ALREADY_EXISTS` 和已有编号；错误 `INVALID_ARGUMENTS`、`COURSE_NOT_FOUND`、`POINT_NOT_FOUND`、`INVALID_DATE`、`USER_DENIED`、`CONFIRM_UNAVAILABLE`、`DATABASE_UNAVAILABLE`、`TOOL_TIMEOUT`

工具白名单、参数检查和访问范围都在工具执行前由程序完成: `ToolRegistry.require` 只接受注册表中的 4 个名称，`validate` 用 Pydantic 模型校验字段类型、必填项、范围和未知字段（`extra=forbid`），通过后才调用工具处理函数；处理函数内部再按当前用户解析课程、知识点，查询使用固定列和参数绑定，行数最多 50 条。写操作只有 `create_review_task` 一个，是否修改数据、等待时间和重复调用行为见表 6-5。

表 6-5: 工具超时、重试、修改数据与重复调用行为

工具	超时	失败重试	修改数据	重复调用结果
`list_courses`	3 秒	最多 2 次	否	无副作用，结果相同
`query_tasks`	3 秒	最多 2 次	否	无副作用，结果相同
`task_stats`	3 秒	最多 2 次	否	无副作用，结果相同
`create_review_task`	5 秒	0 次（写操作不自动重试）	是，插入 `daily_tasks`	按“课程 + 标题 + 日期”去重，同一参数第二次返回 `ALREADY_EXISTS`，不会重复插入；每次调用仍需确认

接入 Agent 前先直接测试工具：正确参数、越界参数、类型错误、课程不存在、未知工具、写操作参数错误和访问范围错误，共 9 次调用。图 6-2 和图 6-3 是实际运行输出，表 6-6 是逐项结果。

```
$ cd backend && python3 -m app.tools.agent_seed --reset >/dev/null 2>&1 && python3 -m app.tools.agen
t_tools_demo 2>&1 | python3 ../tmp/wrapout.py 96 | head -n 12
工具白名单： list_courses、 query_tasks、 task_stats、 create_review_task

[1 正确参数] list_courses({})
  <- ok: 返回 2 门课程： 软件工程、 高等数学
[2 正确参数] task_stats({"course": "软件工程", "days": 14})
  <- ok: 完成率 41.7%（ 5/12）， 薄弱点： 软件测试(30)、 数据库设计(38)
[3 正确参数] query_tasks({"course": "软件工程", "days": 14, "status": "todo"})
  <- ok: 返回 7 条任务（ 已完成 0， 未完成 7）
[4 错误参数： 超出范围] task_stats({"course": "软件工程", "days": 90})
  <- 程序拒绝： INVALID_ARGUMENTS: 参数校验失败： days: Input should be less than or equal to 30
     提示： 请按工具 Schema 修正参数后重试， 程序没有执行该工具
[5 错误参数： 类型错误] query_tasks({"course": "软件工程", "days": "十四"})


$
```

图 6-2: 直接测试一：3 个正确参数调用与第 1 个错误参数调用



图 6-3: 直接测试二：越界、类型、课程、未知工具和写操作参数错误均被拒绝

表 6-6: 接入 Agent 前的工具直接测试

序号	输入	预期	实际结果（判断）
1	`list_courses()`	返回课程	返回 2 门课程：软件工程、高等数学（正确）
2	`task_stats(days=14)`	完成率 41.7%	完成率 41.7%（5/12），薄弱点软件测试 (30)、数据库设计 (38)（正确）
3	`query_tasks(status=todo)`	返回 7 条未完成任务	返回 7 条任务（已完成 0，未完成 7）（正确）
4	`task_stats(days=90)`	校验失败，不执行	`INVALID_ARGUMENTS`：days 应小于等于 30（正确）
5	`query_tasks(days="十四")`	类型错误	`INVALID_ARGUMENTS`：days 不是整数（正确）
6	`query_tasks(course=不存在)`	课程不存在	`COURSE_NOT_FOUND`，提示可用课程（正确）
7	`run_sql(sql=DELETE ...)`	未知工具，不执行	`UNKNOWN_TOOL`，提示白名单（正确）
8	`create_review_task(minutes=99)`	参数越界，不写入	`INVALID_ARGUMENTS`：minutes 应小于等于 60（正确）
9	`create_review_task(point=不存在)`	访问范围错误，不写入	`POINT_NOT_FOUND`，列出课程知识点（正确）

测试结束后任务数仍为 12，说明错误调用没有写入数据。第 8、9 项虽然调用的是写工具，但参数校验或知识点检查在写入前就失败，因此不需要确认也没有产生修改。

3.3 Agent 循环与状态

Agent 循环如图 6-4。程序接收目标和限制后组装上下文（系统提示、4 个工具的 Schema、历史消息），调用模型；模型返回工具调用请求时，程序先做工具白名单、参数 Schema 和访问范围检查，写操作还要经过人工确认，然后执行工具，把“assistant 的 tool_calls + 对应 tool 结果”按 `tool_call_id` 写回消息列表，再进入下一轮；模型返回最终回答时，程序直接查数据库核对成功标准，不通过则带着差异反馈给模型做有限次修正。图中的红色横杠标出了停止检查的位置：每次模型调用和每次工具执行之前都检查步数上限与总运行时限。

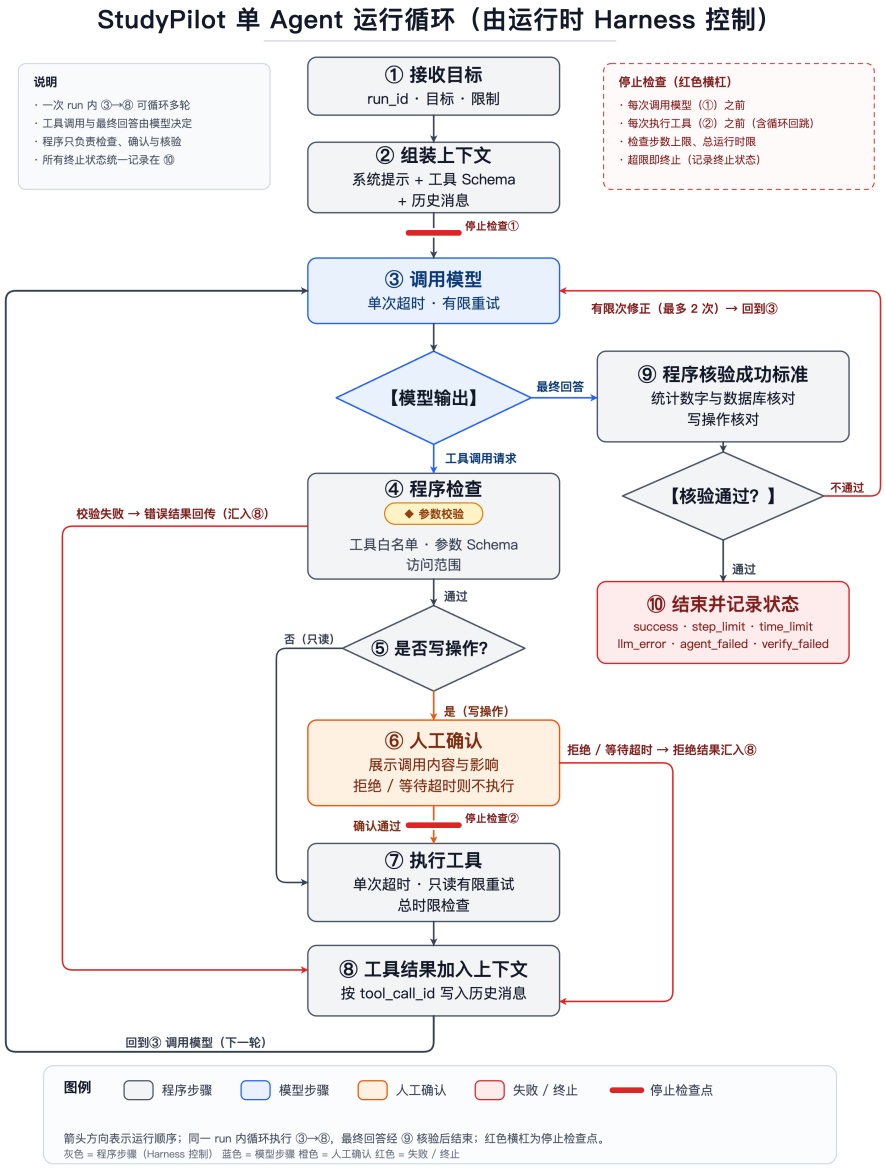


图 6-4: StudyPilot 单 Agent 运行循环与 Harness 控制点

本实验把“一步”定义为一次模型请求；同一次请求中的工具调用属于这一步，工具内部的只读重试不单独计步，但会记录尝试次数；最终回答格式不合法或数字核对不通

过时，把反馈作为下一次模型请求的输入，因此一次修正计一步。各项限制及超限结果见表 6-7，全部由 AgentHarness、ToolRegistry 或命令行确认函数执行，而不是只写在提示词里。

表 6-7: 运行限制、执行位置与超限结果

限制	取值	执行位置	超限结果
最大执行步数	8 步	harness.run 每轮循环开头检查	状态 step_limit，写明“达到最大执行步数”
总运行时限	90 秒	每次模型/工具调用前用剩余时间限制超时	状态 time_limit
模型单次超时	30 秒	llm_client.chat_tools 的 httpx 超时	可重试错误重试 1 次，仍失败则 llm_error
模型重试上限	1 次	_call_model	记录 llm_retry 后失败
工具单次超时	只读 3 秒，写 5 秒	Registry.execute 线程等待	TOOL_TIMEOUT；只读可重试，写不重试
工具重试上限	只读 2 次	_handle_tool_call	错误结果回传模型
每步工具调用数	最多 3 个	harness.run 循环	多余调用返回 TOO_MANY_TOOL_CALLS
结果修正次数	最多 2 次	harness.run 最终回答分支	状态 verify_failed
确认等待时间	20 秒（且不超过剩余总时限）	make_confirm	视为拒绝，不执行写操作；等待时间计入总时限

图 6-5 是一次完整的真实运行：模型在第 1 步确认课程，第 2 步取统计结果，第 3 步判断完成率 41.7% 低于 60% 后请求创建复习任务，程序展示参数和影响并等待确认，用户输入 y 后写入任务 #486，第 4 步模型给出 JSON 最终回答，程序 8 项核验全部通过。本次共 4 步、4 次模型调用、3 次工具调用，耗时 5772 毫秒，累计 tokens=7171。



图 6-5: 正常完成并同意写操作：4 步、任务 #486、8 项核验通过（run_id 20261109-104557-af8bf4）

工具调用和结果确实进入了后续上下文：从日志中的 `messages` 字段可以看到上下文条数依次为 2（系统 + 目标）、4（加入 assistant tool_calls 和 tool 结果）、6、8；每次请求都带上之前的完整消息。代码上，assistant 消息保留 `tool_calls`，工具结果写成 `{"role":"tool","tool_call_id":...,"content":...}`，而不是只打印到终端。提示词只说明任务规则和最终 JSON 格式，不负责执行限制；例如即使模型要求第 9 步，程

序也会在第 8 步后直接终止。

最终回答由 verifier.ground_truth 独立查询数据库得到基准值，再由 verify_success 逐项比较：完成率、任务总数、已完成数、未完成数、薄弱知识点、建议是否为空、条件写操作是否与数据库一致、最终状态字段。图 6-6 演示了核验生效的过程：脚本模型先给出错误的完成率 50.0，程序在 verify 事件中判为不通过并把差异反馈给模型，第 5 步模型改成 41.7 后才通过；如果两次修正仍不通过，运行会以 verify_failed 结束，而不是因为模型声称完成就判成功。

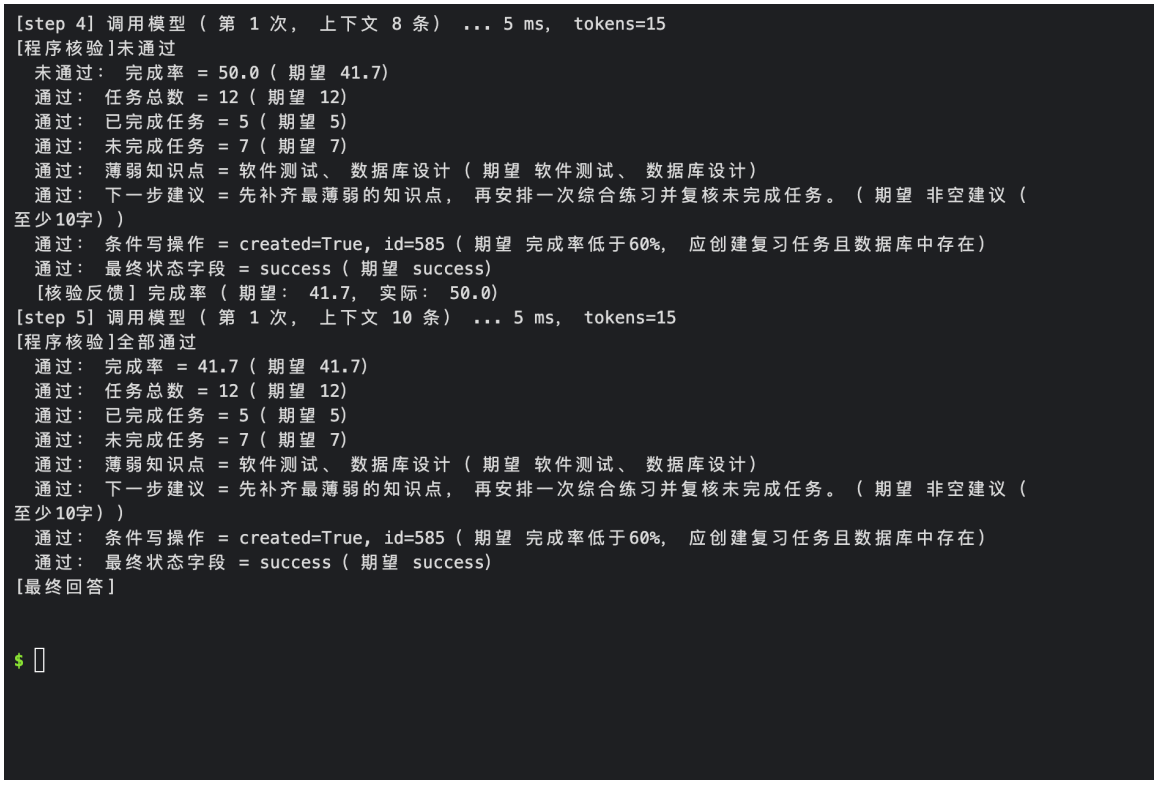


图 6-6: 结果核验：错误完成率 50.0 被程序拒绝，修正为 41.7 后通过（run_id 20261109-104611-b115fc）

除数字核对外，核验还检查写操作的一致性：模型声称创建时数据库必须有对应任务；声称未创建时本次运行不能有实际写入，避免“实际写了却回答 false”的漏判。

3.4 运行时 Harness 设计与实现

运行时 Harness 由 backend/app/agent/ 下的模块组成，模块分工见表 6-8。不拆成独立服务，命令行入口和测试都直接调用同一个 AgentHarness。

表 6-8: Harness 模块分工（任务输入、上下文、执行、结果检查、日志）

模块	职责	关键函数/类
agent/prompts.py	任务规则、工具使用约束、最终 JSON 格式；不承担强制限制	AGENT_SYSTEM_PROMPT
agent/spec.py	工具规格（名称、Schema、处理函数、超时、是否写、影响说明）和统一失败类型	ToolSpec、ToolFailure
agent/tools.py	4 个工具的实现、参数模型、数据库访问范围、写操作去重	list_courses、query_tasks、task_stats、create_review_task、TOOL_SPECS
agent/registry.py	工具白名单、参数校验、线程超时执行	require、validate、execute
agent/harness.py	任务输入、上下文组装、主循环、步数/时限/重试/确认、结果反馈、日志	AgentHarness.run、_call_model、_handle_tool_call
agent/verifier.py	独立查库得到成功标准基准，逐项核验最终回答和写操作	ground_truth、verify_success
agent/runlog.py	JSONL 运行日志、密钥遮挡、结果摘要	RunLogger、mask_text
agent/scenario.py	可重复的 12 条任务样例数据	reset_scenario
service/llm_client.py	DeepSeek chat completions 工具调用，返回 assistant 消息和用量	chat_tools
tools/agent_cli.py	命令行入口、确认交互（ask/yes/no，20 秒超时）	make_confirm、main
tools/agent_log.py	按运行编号查看 JSONL 日志	main
tests/test_agent.py	23 个离线用例：工具、循环、限制、确认、核验、日志	见 3.6

控制流程可以概括为：AgentHarness.run 接收目标、课程、天数和限制；_call_model 把当前上下文和工具 Schema 发给 chat_tools；模型返回后，_handle_tool_call 先做白名单和参数校验，写操作调用确认回调，然后由 ToolRegistry.execute 在单独线程中执行并等待超时；结果以 tool 消息回传；模型给出最终回答时调用 verifier 核验。工具失败时，错误码、消息和修正提示作为 ok=false 的结果回传，模型可以选择修正参数、换只读工具核对或报告失败；核验不通过时同理，最多 2 次。日志在每个关键点写入 backend/logs/agent_runs.jsonl。

3.5 失败处理与可观测性

每次运行生成形如 20261109-104557-af8bf4 的运行编号，日志按事件记录时间、步骤、工具与参数摘要、结果摘要、错误码、耗时、tokens 和最终状态。主要字段见表 6-9，实际日志见图 6-7。

表 6-9: 运行日志事件与字段

事件	主要字段
`run_start`	run_id、时间、目标、课程、天数、步数/时限/重试/修正上限、工具白名单
`model_call`	步骤、第几次尝试、耗时、上下文消息数
`model_reply`	步骤、耗时、tokens、请求的工具名、内容摘要（截断 200 字）
`tool_request`	步骤、call_id、工具名、参数摘要
`tool_result` / `tool_retry` / `tool_rejected`	工具名、是否成功、尝试次数、耗时、结果摘要或错误码/消息
`confirm`	工具、参数、approved、拒绝原因、等待毫秒数
`verify`	步骤、是否通过、逐项期望值与实际值
`run_end`	状态、步数、模型/工具调用次数、总耗时、tokens、错误、创建的任务编号、拒绝过的工具、日志路径



图 6-7: 一次真实运行的 JSONL 日志摘要：上下文条数 2/4/6/8，工具结果、确认、核验和结束状态（run_id 20261109-104557-af8bf4）

各类失败的处理方式见表 6-10。参数错误在工具执行前被拦下，不会产生修改；只读工具异常按上限重试后把错误回传；超时只停止等待，线程不能强制终止；写操作超时状态可能不明，程序不自动重试，提示模型先用只读查询核对；人工确认被拒绝或超时则不执行。

表 6-10: 失败情况、状态与处理

情况	错误码/状态	处理	证据
参数错误	INVALID_ARGUMENTS	校验失败，工具不执行，错误和修正提示回传模型；模型修正后继续	图 6-9
未知工具	UNKNOWN_TOOL	白名单拒绝，不执行任意函数名	图 6-3
课程/知识点不存在	COURSE_NOT_FOUND / POINT_NOT_FOUND	访问范围检查失败，不写入	图 6-3
工具异常（数据库损坏）	DATABASE_UNAVAILABLE	只读重试 2 次后回传错误，模型报告失败，状态 agent_failed	图 6-11
工具超时	TOOL_TIMEOUT	只停止等待；只读可重试，写操作不重试并提示先核对	限制表 6-7
达到步数上限	step_limit	第 8 步后终止并写明原因	图 6-12
达到时间上限	time_limit	超过 90 秒终止，不再开始新的工具调用	图 6-13
模型调用失败	LLM_* / llm_error	可重试错误重试 1 次，仍失败则结束	日志字段
人工拒绝/确认超时	USER_DENIED	工具不执行，按未创建任务收尾，不再次请求写操作	图 6-8
最终回答不合法/核验不通过	verify_failed	反馈差异，最多修正 2 次，仍失败则结束并保留检查结果	图 6-6
模型自报失败	agent_failed	接受“failed”JSON 并记录原因，不伪造成功	图 6-11

关于超时的取消能力：工具在单独线程中执行，future.result(timeout) 超时后程序停止等待，但 Python 线程本身不能被强制终止。三个只读工具即使后台继续跑完也不会修改数据；写工具 create_review_task 超时后状态可能不明，因此 Harness 对写操作不自动重试，返回的提示要求模型先用 query_tasks 核对是否已经写入，再决定是否向用户说明，符合“状态不明先核对、不得直接重试”的要求。模型调用的超时通过 httpx 的 timeout 参数控制，超时后可以重试一次。

人工确认在写操作执行前展示工具名、完整参数和影响说明，并等待用户输入；确认同意后才调用工具，拒绝或 20 秒无输入则不执行。图 6-8 是同一任务在用户拒绝时的运行：模型请求创建后程序询问，用户拒绝，工具返回 USER_DENIED，第 4 步模型给出 review_task_created=false，程序核验通过且数据库没有新增任务。确认等待时间计入总运行时限（取剩余时间与 20 秒的较小值）；确认返回后如果时限已到，即使同意也不执行写操作。同一运行中用户拒绝过的写操作再次被模型请求时，程序不再询问，直接返回 WRITE_ALREADY_DENIED，防止反复打扰用户。



图 6-8: 人工确认被拒绝：写操作不执行，最终回答如实说明未创建（run_id 20261109-104603-93eeb3）

密钥只从环境变量 DEEPSEEK_API_KEY 读取，不写进代码、命令行参数和日志；runlog 对字符串中的 sk- 前缀密钥再做一次遮挡。日志记录参数摘要和结果摘要，不记录完整系统提示和密钥；课程名、任务标题属于本人样例数据，不含真实用户隐私。

3.6 测试与结果分析

测试分两部分：23 个离线单元测试覆盖工具、循环、限制、确认、核验和日志，使用临时数据库和脚本模型，不调用外部模型；正常完成、人工确认同意和拒绝另外用真实模型和真实工具各跑一次（图 6-5、图 6-8），参数错误、工具异常、步数/时间限制和核验失败用脚本模型模拟模型决策，但工具、确认和 Harness 都是真实实现。测试分类与结果见表 6-11，离线测试共 43 个用例（含实验 4、实验 5 的 20 个），全部通过。

表 6-11: 测试情况：输入、预期、实际与判断依据

编号	类别	输入	预期	实际结果	判断
T1	正常完成	真实模型 + 真实工具，目标为统计 14 天并条件创建	4 步内完成；创建任务；8 项核验通过	4 步、3 次工具调用、新建 #486、核验全通过、tokens=7171	符合；图 6-5
T2	参数错误	脚本模型先请求 `query_tasks` 的 days=90，再正常调用	程序拒绝且不执行，错误回传，模型修正后成功	第 1 步 `INVALID_ARGUMENTS`，第 2—5 步修正并完成	符合；图 6-9
T3	工具异常	把数据库指向损坏文件，脚本模型请求 `task_stats`	只读重试 2 次后报错，模型报告失败，不伪造统计	3 次尝试后 `DATABASE_UNAVAILABLE`，状态 `agent_failed`，原因明确	符合；图 6-11
T4	步数上限	脚本模型反复请求 `list_courses`，max_steps=3	第 3 步后终止，状态 `step_limit`	3 步、3 次工具调用后终止，原因“达到最大执行步数 3”	符合；图 6-12
T5	时间上限	脚本模型调用耗时 1.5 秒，总时限 1 秒	超时终止，状态 `time_limit`	1 步、耗时 1509 毫秒后终止，工具未执行	符合；图 6-13
T6	确认同意	真实模型请求 `create_review_task`，输入 y	展示参数与影响，写入任务，最终回答 true	输入 y 后创建任务 #486，数据库可查到，核验通过	符合；图 6-5
T7	确认拒绝	真实模型请求 `create_review_task`，–confirm no	不执行，最终回答 false，数据库无新任务	返回 `USER_DENIED`，最终回答 false，核验通过	符合；图 6-8
T8	结果核验	脚本模型先给完成率 50.0，再给 41.7	第一次不通过并反馈，修正后通过	第 4 步 `verify` 未通过，第 5 步修正后全部通过	符合；图 6-6
T9	工具超时	构造 0.3 秒慢工具，超时设 0.05 秒	抛 `TOOL_TIMEOUT`，只读错误可重试	0.05 秒后抛出，`retriable=true`	符合；单元测试
T10	确认等待超时	`make_confirm` 的 select 无输入	返回拒绝，不执行写操作	`approved=false`，原因“确认等待超时”	符合；单元测试
T11	离线回归	`pytest tests -q`	全部通过	43 passed（含 23 个 Agent 用例）	符合；图 6-14



图 6-9: 参数错误：days=90 被程序拒绝，错误回传后模型改用合法参数（run_id 20261109-104609-0e8b37）

图 6-10 是同一运行的后半段：确认同意后创建任务 #511，程序核验通过并输出 `status=success`；完整输出保存在 `reports/experiment6/runs/e6-param_error.txt`。

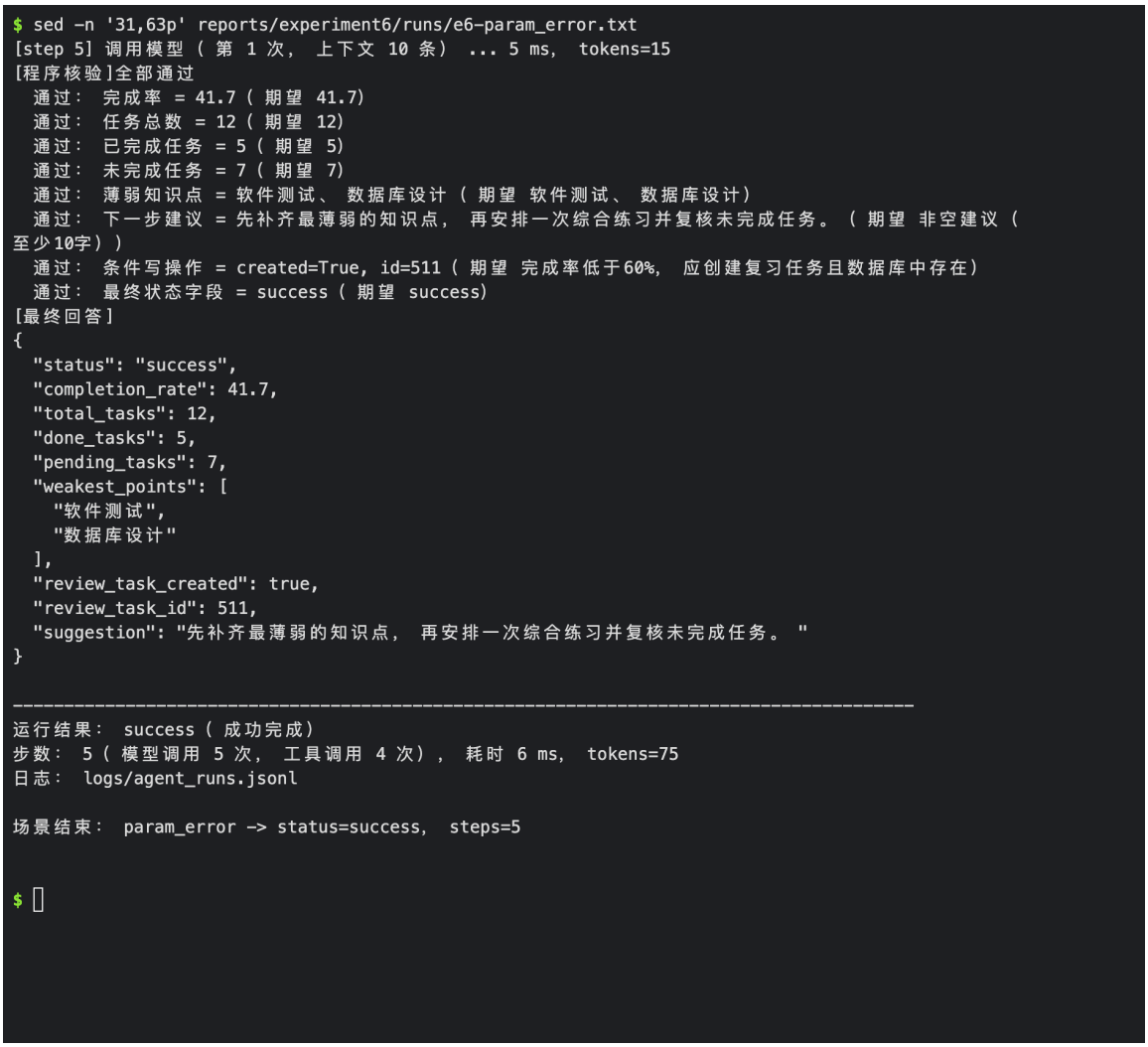


图 6-10: 参数错误场景后半段：确认、程序核验与最终成功状态（同一 run_id 20261109-104609-0e8b37）



图 6-11: 工具异常: 数据库不可用, 只读工具重试 2 次后失败, 模型如实报告失败（run_id 20261109-104609-7f46a6）



图 6-12: 达到最大步数：第 3 步后终止，状态 step_limit（run_id 20261109-104609-e21d52）

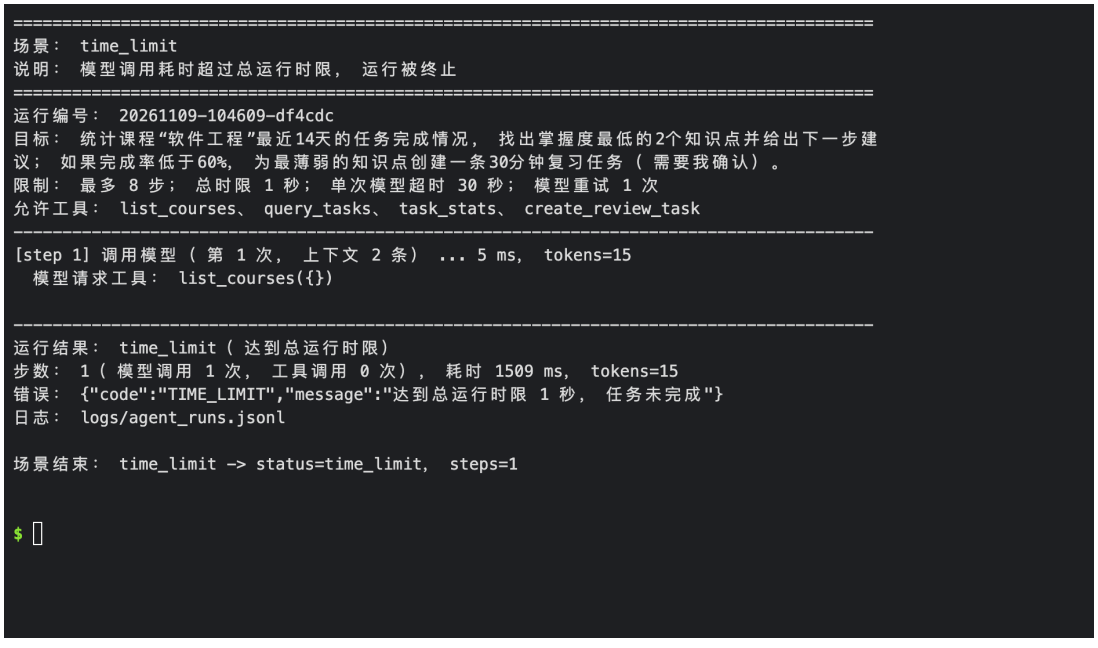


图 6-13: 达到总运行时限：模型调用耗时超过 1 秒后终止，工具未执行（run_id 20261109-104609-df4cdc）

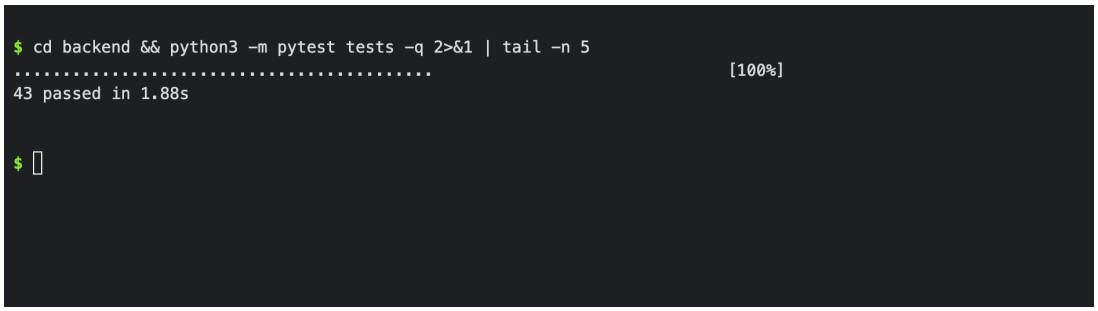


图 6-14: 离线测试：43 个用例全部通过

从结果看，正常流程可以稳定完成：真实模型在 4 步内按“确认课程 → 统计 → 条件创建 → 最终回答”的顺序执行，数字与数据库一致，写操作前确实停下等待确认；拒绝分支没有产生写入，最终回答也如实说明未创建。参数错误和未知工具都在执行前被拦下，错误信息能指导模型修正；工具异常时只读工具按上限重试后停止，模型没有编造统计结果。步数和时间限制由程序触发，日志中的状态和原因清楚。程序核验能抓住模型声称与数据不符的情况，图 6-6 中的 50.0 就是被核验拦下的；如果没有这道检查，模型只要输出“success”就会被误判为完成。

也有需要说明的地方。第一，异常和确认场景用脚本模型模拟“模型决策”，这样能稳定复现，但不是模型真实生成的行为；真实模型只验证了正常、同意和拒绝三条路径。第二，脚本模型给出的最终回答由测试代码从数据库基准值构造，因此这些场景考察的是 Harness 控制流和核验，不是模型能力。第三，步数和时间限制场景中脚本模型故意重复请求工具或超时，真实模型在 90 秒内通常不会触发这些边界；因此边界处理依靠程序逻辑和单元测试保证。第四，真实运行中模型会自行在最终建议里补充“6 条逾期任务”等细节，这些数字来自工具结果，程序只核验约定的字段，没有对建议文本逐句核查。

当前局限：一是任务和样例数据固定在一门课程，工具只支持最近 30 天的任务查询，不能处理多课程汇总；二是工具超时只能停止等待，后台线程可能继续运行，写操作超时后的状态需要人工或只读查询核对；三是最终回答要求固定 JSON 格式，模型偶尔会多写解释文字，本次通过一次提示可以纠正，但会增加步数；四是确认只在命令行实现，尚未接入网页；五是日志只追加写文件，没有日志轮转和检索界面；六是真实模型调用仍可能受网络延迟影响，本实验用 30 秒单次超时和 90 秒总时限兜底。

下一步计划：把 Agent 接入 StudyPilot 的页面和接口，提供“分析本周学习情况”按钮和确认对话框；增加 `complete_task`、`adjust_plan` 等工具，并把写操作统一做成可审计的待确认动作；把日志写入数据库并加运行列表页；用固定的 10 组目标做回归测试，比较不同提示词和工具集的成功率、步数与耗时；对长时间运行的工具使用可取消进程或异步任务，补上真正的取消能力；再考虑多课程汇总和简单的向量检索辅助建议。

Agent 和 Harness 已用于个人软件的样例数据库：工具读写的是 StudyPilot 自己的 `studypilot.db`，写操作写入 `daily_tasks`，与实验 4 的学习计划、实验

5 的资料问答共用同一套数据和日志规范；目前只在命令行入口使用，页面接入列在下一步。代码位于 backend/app/agent/、backend/app/tools/agent_*.py、backend/tests/test_agent.py，运行说明见仓库 README.md，对应 commit 见 3.8 节。

3.7 关键代码

代码直接从项目文件提取，省略了与本实验无关的辅助函数。

```python
class QueryTasksArgs(BaseModel):
    model_config = ConfigDict(extra="forbid")

    course: str = Field(..., min_length=1, max_length=30, description="课程名称，例如“软件工程”")
    days: int = Field(14, ge=1, le=30, description="查询最近多少天（含今天），取值 1—30")
    status: Literal["all", "todo", "done"] = Field("all", description="任务状态过滤：all/todo/done")


class TaskStatsArgs(BaseModel):
    model_config = ConfigDict(extra="forbid")

    course: str = Field(..., min_length=1, max_length=30, description="课程名称，例如“软件工程”")
    days: int = Field(14, ge=1, le=30, description="统计最近多少天（含今天），取值 1—30")


class CreateReviewTaskArgs(BaseModel):
    model_config = ConfigDict(extra="forbid")

    course: str = Field(..., min_length=1, max_length=30, description="课程名称，例如“软件工程”")
    point: str = Field(..., min_length=1, max_length=30, description="知识点名称，必须属于该课程")
    minutes: int = Field(30, ge=10, le=60, description="复习时长（分钟），取值 10—60")
    date: str | None = Field(None, description="复习日期 YYYY-MM-DD，缺省为明天")

    @field_validator("date")
    @classmethod
    def _check_date(cls, value: str | None) -> str | None:
        if value is None:
            return None
        try:
            datetime.date.fromisoformat(value)
        except ValueError as exc:
            raise ValueError("日期格式应为 YYYY-MM-DD") from exc
        return value
```

代码 6-1: 工具参数模型：类型、范围、必填项与未知字段限制（backend/app/agent/tools.py 第 67—101 行）

```python
    def validate(self, name: str, raw_args: object) -> tuple[ToolSpec, BaseModel]:
        spec = self.require(name)
        if not isinstance(raw_args, dict):
            raise ToolFailure("工具参数必须是 JSON 对象", code="INVALID_ARGUMENTS",
                              hint="按工具 Schema 提供参数对象")
        try:
            parsed = spec.args_model.model_validate(raw_args)
        except ValidationError as exc:
```

```
            details = []
            for err in exc.errors():
                location = ".".join(str(part) for part in err["loc"]) or "参数"
                details.append(f"{location}: {err['msg']}")
            raise ToolFailure(
                "参数校验失败: " + "; ".join(details),
                code="INVALID_ARGUMENTS",
                hint="请按工具 Schema 修正参数后重试，程序没有执行该工具",
            ) from exc
        return spec, parsed

    def execute(self, spec: ToolSpec, parsed: BaseModel, timeout_s: float) -> dict:
        """在单独线程中执行工具，超过 timeout_s 时停止等待并抛出 TOOL_TIMEOUT。

        注意：线程无法被强制终止，读工具在后台继续执行没有副作用；写工具超时后
        状态可能不明，Harness 不会自动重试，而是要求先查询核对。
        """
        started = time.monotonic()
        executor = concurrent.futures.ThreadPoolExecutor(
            max_workers=1, thread_name_prefix=f"tool-{spec.name}"
        )
        try:
            future = executor.submit(spec.handler, parsed)
            try:
                data = future.result(timeout=timeout_s)
            except concurrent.futures.TimeoutError as exc:
                raise ToolFailure(
                    f"工具执行超过 {timeout_s:.1f} 秒仍未返回",
                    code="TOOL_TIMEOUT",
                    retriable=not spec.mutating,
                    hint=("读操作可以重试；写操作超时后状态未知，必须先查询核对，不能直接重试"
                          if spec.mutating else "可以稍后重试"),
                ) from exc
        except ToolFailure:
            raise
        except Exception as exc:  # noqa: BLE001 - 统一转成工具错误回传模型
            raise ToolFailure(
                f"工具执行异常: {exc.__class__.__name__}",
                code="TOOL_EXCEPTION",
                retriable=False,
                hint="该错误不可自动重试，请向用户说明失败原因",
            ) from exc
        finally:
            executor.shutdown(wait=False, cancel_futures=True)
        return {"data": data, "latency_ms": int((time.monotonic() - started) * 1000)}
```

代码 6-2: 工具白名单、参数校验和带超时的执行（backend/app/agent/registry.py 第 33—85 行）

```
            if step >= self.max_steps:
                status = STATUS_STEP_LIMIT
                error = {"code": "STEP_LIMIT",
                         "message": f"达到最大执行步数 {self.max_steps}，任务未完成"}
                break
            if self._remaining() <= 0:
                status = STATUS_TIME_LIMIT
```

```
                error = {"code": "TIME_LIMIT",
                         "message": f"达到总运行时限 {self.deadline_s:.0f} 秒，任务未完成"}
                break

            step += 1
            try:
                response = self._call_model(messages, step)
            except BusinessError as exc:
                if self._remaining() <= 0 or exc.code == "TIME_LIMIT":
                    status = STATUS_TIME_LIMIT
                    error = {"code": "TIME_LIMIT",
                             "message": f"达到总运行时限 {self.deadline_s:.0f} 秒（{exc.message}）"}
                else:
                    status = STATUS_LLM_ERROR
                    error = {"code": exc.code, "message": exc.message}
                self._emit("llm_error", step=step, code=error["code"], message=error["message"])
                break

            usage = response.get("usage") or {}
            self.tokens += int(usage.get("total_tokens") or 0)
            self.model_calls += 1
            message = response.get("message") or {}
            content = message.get("content") or ""
            tool_calls = message.get("tool_calls") or []
            self.logger.event(
                "model_reply", step=step, latency_ms=response.get("latency_ms"),
                tokens=usage.get("total_tokens"),
                tool_calls=[(c.get("function") or {}).get("name") for c in tool_calls],
                content=summarize(content, 200),
            )
            self._emit(
                "model_reply", step=step, latency_ms=response.get("latency_ms"),
                tokens=usage.get("total_tokens"), content=content,
                tool_calls=[{"name": (c.get("function") or {}).get("name"),
                             "arguments": (c.get("function") or {}).get("arguments")}
                            for c in tool_calls],
            )

            if self._remaining() <= 0:
                status = STATUS_TIME_LIMIT
                error = {"code": "TIME_LIMIT",
                         "message": f"达到总运行时限 {self.deadline_s:.0f} 秒，任务未完成"}
```

代码 6-3: Harness 主循环中的步数与总时限检查（backend/app/agent/harness.py 第 244—292 行）

```
            # ---- 最终回答：先解析，再由程序核验 ----
            parsed, parse_error = _parse_final_content(content)
            if parse_error:
                corrections += 1
                self.logger.event("final_rejected", step=step, error="INVALID_FINAL", message=
    parse_error)
                self._emit("final_rejected", step=step, error="INVALID_FINAL", message=parse_error)
                if corrections > self.max_corrections:
                    status = STATUS_VERIFY_FAILED
                    error = {"code": "INVALID_FINAL",
```

```
                        "message": f"最终回答不是约定的 JSON，已修正 {self.max_corrections} 次仍未通过：{parse_error}"}
                break
            messages.append({"role": "assistant", "content": content})
            messages.append({"role": "user", "content":
                             f"你的输出不是约定的 JSON（{parse_error}）。请只输出 JSON 对象，"
                             "不要 Markdown 代码围栏或其他文字。"})
            continue

        if parsed.get("status") == "failed":
            status = STATUS_AGENT_FAILED
            final_answer = parsed
            error = {"code": "AGENT_REPORTED_FAILURE",
                     "message": str(parsed.get("reason") or "模型报告无法完成任务")}
            break

        try:
            gt = ground_truth(course, days)
        except BusinessError as exc:
            status = STATUS_VERIFY_FAILED
            error = {"code": exc.code, "message": f"无法读取数据库核对结果：{exc.message}"}
            self._emit("verify_error", step=step, message=error["message"])
            break

        checks = verify_success(parsed, gt, {
            "created_task_ids": self.created_task_ids,
            "denied_tools": self.denied_tools,
        })
        passed = all(check["ok"] for check in checks)
        self.logger.event("verify", step=step, passed=passed,
                          checks=[{k: c[k] for k in ("name", "ok", "expected", "actual")} for c in checks])
        self._emit("verify", step=step, passed=passed, checks=checks,
                   ground_truth=gt)
        if passed:
            status = STATUS_SUCCESS
            final_answer = parsed
            break

        corrections += 1
        failed = [c for c in checks if not c["ok"]]
        detail = "; ".join(
            f"{c['name']}（期望：{c['expected']}，实际：{c['actual']}）" for c in failed
        )
        if corrections > self.max_corrections:
            status = STATUS_VERIFY_FAILED
            final_answer = parsed
            error = {"code": "VERIFY_FAILED",
                     "message": f"程序核验不通过，已修正 {self.max_corrections} 次仍未通过：{detail}"}
            break
        self.logger.event("final_rejected", step=step, error="VERIFY_FAILED", message=detail)
        self._emit("final_rejected", step=step, error="VERIFY_FAILED", message=detail)
        messages.append({"role": "assistant", "content": content})
        messages.append({"role": "user", "content":
                         f"程序核验未通过：{detail}。请调用工具核对数据，并只输出修正后的 JSON。"})
```

代码 6-4: 最终回答解析、程序核验与有限次修正（backend/app/agent/harness.py 第 333—395 行）

```python
def verify_success(answer: dict, gt: dict, context: dict | None = None) -> list[dict]:
    """返回逐项核验结果；全部通过才算任务完成。"""
    context = context or {}
    created_ids = context.get("created_task_ids") or []
    denied = context.get("denied_tools") or []
    checks: list[dict] = []

    def add(name: str, ok: bool, expected, actual, detail: str = ""):
        checks.append({"name": name, "ok": bool(ok), "expected": expected, "actual": actual, "detail": detail})

    rate = _number(answer.get("completion_rate"))
    add("完成率", rate is not None and round(rate, 1) == gt["completion_rate"],
        gt["completion_rate"], answer.get("completion_rate"))

    for field, label in (("total_tasks", "任务总数"), ("done_tasks", "已完成任务"), ("pending_tasks", "未完成任务")):
        value = answer.get(field)
        add(label, isinstance(value, int) and value == gt[field], gt[field], value)

    names = _point_names(answer.get("weakest_points"))
    add("薄弱知识点", set(names) == set(gt["weakest_points"]),
        "、".join(gt["weakest_points"]), "、".join(names) or "空")

    suggestion = answer.get("suggestion")
    add("下一步建议", isinstance(suggestion, str) and len(suggestion.strip()) >= 10,
        "非空建议（至少10字）", suggestion)

    claimed = answer.get("review_task_created")
    claimed_id = answer.get("review_task_id")
    if gt["completion_rate"] < 60:
        if claimed is True:
            ok = bool(created_ids) and claimed_id in created_ids and _task_exists(claimed_id)
            add("条件写操作", ok, "完成率低于60%，应创建复习任务且数据库中存在",
                f"created={claimed}, id={claimed_id}")
        else:
            ok = "create_review_task" in denied and not created_ids
            add("条件写操作", ok, "完成率低于60%，创建复习任务或记录用户拒绝，且本次无写入",
                "用户已拒绝/确认超时，未创建" if ok else "声称未创建，但存在写入或没有拒绝记录")
    else:
        ok = claimed is not True and not created_ids
        add("条件写操作", ok, "完成率不低于60%，不应创建写任务",
            f"created={claimed}, ids={created_ids}")
```

代码 6-5: 结果核验：逐项对照数据库基准值（backend/app/agent/verifier.py 第 89—130 行）

```python
        if spec.mutating:
            if spec.name in self.denied_tools:
                exc = ToolFailure("该写操作此前已被用户拒绝，本次运行不再请求确认，也不会执行",
                                  code="WRITE_ALREADY_DENIED",
                                  hint="按未创建任务继续给出最终回答，不要再次请求写操作")
```

```
            self.tool_calls += 1
            self.logger.event("tool_rejected", step=step, tool=spec.name,
                              code=exc.code, message=exc.message)
            self._emit("tool_rejected", step=step, tool=spec.name,
                       code=exc.code, message=exc.message)
            return call_id, _error_result(exc)
        if self._remaining() <= 0:
            exc = ToolFailure("总运行时限已到，写操作没有请求确认，也没有执行",
                              code="TIME_LIMIT", hint="本次运行已结束，不执行写操作")
            self.tool_calls += 1
            self.logger.event("tool_rejected", step=step, tool=spec.name,
                              code="TIME_LIMIT", message=exc.message)
            self._emit("tool_rejected", step=step, tool=spec.name,
                       code="TIME_LIMIT", message=exc.message)
            return call_id, _error_result(exc)
        if self.confirm is None:
            exc = ToolFailure("写操作需要人工确认，但当前运行没有确认通道，已拒绝执行",
                              code="CONFIRM_UNAVAILABLE", hint="本次未执行任何写操作")
            self.denied_tools.append(spec.name)
            self.tool_calls += 1
            self.logger.event("confirm", step=step, tool=spec.name, args=args_dict,
                              approved=False, reason="NO_CONFIRM_CHANNEL", waited_ms=0)
            self._emit("confirm", step=step, tool=spec.name, args=args_dict,
                       impact=spec.impact, approved=False, reason="没有确认通道", waited_ms=0)
            return call_id, _error_result(exc)

        request = ConfirmationRequest(
            run_id=self.run_id, step=step, tool=spec.name, args=args_dict,
            impact=spec.impact, timeout_s=min(20.0, max(0.0, self._remaining())),
        )
        self._emit("confirm_request", step=step, tool=spec.name, args=args_dict,
                   impact=spec.impact, timeout_s=request.timeout_s)
        decision = self.confirm(request) or {}
        approved = bool(decision.get("approved"))
        waited_ms = int(decision.get("waited_ms") or 0)
        reason = str(decision.get("reason") or ("用户同意" if approved else "用户拒绝"))
        self.logger.event("confirm", step=step, tool=spec.name, args=args_dict,
                          approved=approved, reason=reason, waited_ms=waited_ms)
        self._emit("confirm", step=step, tool=spec.name, approved=approved,
                   reason=reason, waited_ms=waited_ms)
        if not approved:
            self.denied_tools.append(spec.name)
            self.tool_calls += 1
            return call_id, _error_result(ToolFailure(
                "用户未同意该操作，工具没有执行", code="USER_DENIED",
                hint="不要再次请求写操作，按未创建任务继续给出最终回答",
            ))
        if self._remaining() <= 0:
            self.tool_calls += 1
            self.logger.event("tool_rejected", step=step, tool=spec.name,
                              code="TIME_LIMIT", message="确认等待后总时限已到，写操作未执行")
            self._emit("tool_rejected", step=step, tool=spec.name,
                       code="TIME_LIMIT", message="确认等待后总时限已到，写操作未执行")
            return call_id, _error_result(ToolFailure(
                "确认等待后总运行时限已到，写操作没有执行", code="TIME_LIMIT",
                hint="本次运行已结束，不执行写操作",
```

```
62                 ))
63
64         attempts = 0
65         while True:
```

代码 6-6: 写操作的人工确认、拒绝重复请求与时限检查（backend/app/agent/harness.py 第 492—556 行）

3.8 版本记录

代码提交到 GitHub 仓库 https://github.com/luohao711/StudyPilot。本实验的代码（工具、Agent、Harness、测试和运行说明）对应 commit `63f9490`，tag `v0.4`；报告、截图和运行记录随本实验一并提交。报告中的数据、截图和日志来自本机 2026-11-09 的运行，源文件、循环图、截图和运行记录放在 `reports/experiment6/` 与 `backend/logs/agent_runs.jsonl` 下。

```
$ git log --oneline --decorate -6 | fold -s -w 92; echo; git tag -n1; echo; git show -s --format='or
igin/main = %H' origin/main
63f9490 (HEAD -> main, tag: v0.4, origin/main) 实验6：实现工具调用、单Agent运行时与Harness
bc7887c 实验5：更新GitHub版本记录与最终PDF
6aca873 实验5：最终报告与实拍截图（去除终端页眉标注）
44eb675 实验5：重跑证据、修复知识库索引并更新报告数据
fa07190 实验5：完成报告正文、运行截图与最终PDF
7fe795a (tag: v0.3)
实验5：实现资料问答（模型调用、Prompt迭代、知识库切分、BM25检索与RAG对比测试）

v0.1            实验4可运行初版
v0.2            实验4测试修订版
v0.3            实验5 RAG资料问答
v0.4            实验6：工具调用、Agent与Harness

origin/main = 63f94908fad7044edd65df860e549efb43e324cd


$
```

图 6-15: 版本记录：代码提交与 tag v0.4

```
$ git push origin main 2>&1; git push origin v0.4 2>&1; echo; git ls-remote --heads --tags origin |
grep -E 'refs/heads/main|v0.4'
Everything up-to-date
Everything up-to-date

63f94908fad7044edd65df860e549efb43e324cd        refs/heads/main
4531ec4262e467465f865093288474423fdef214        refs/tags/v0.4
63f94908fad7044edd65df860e549efb43e324cd        refs/tags/v0.4^{}


$
```

图 6-16: 推送后校验：远端 main 与本地一致

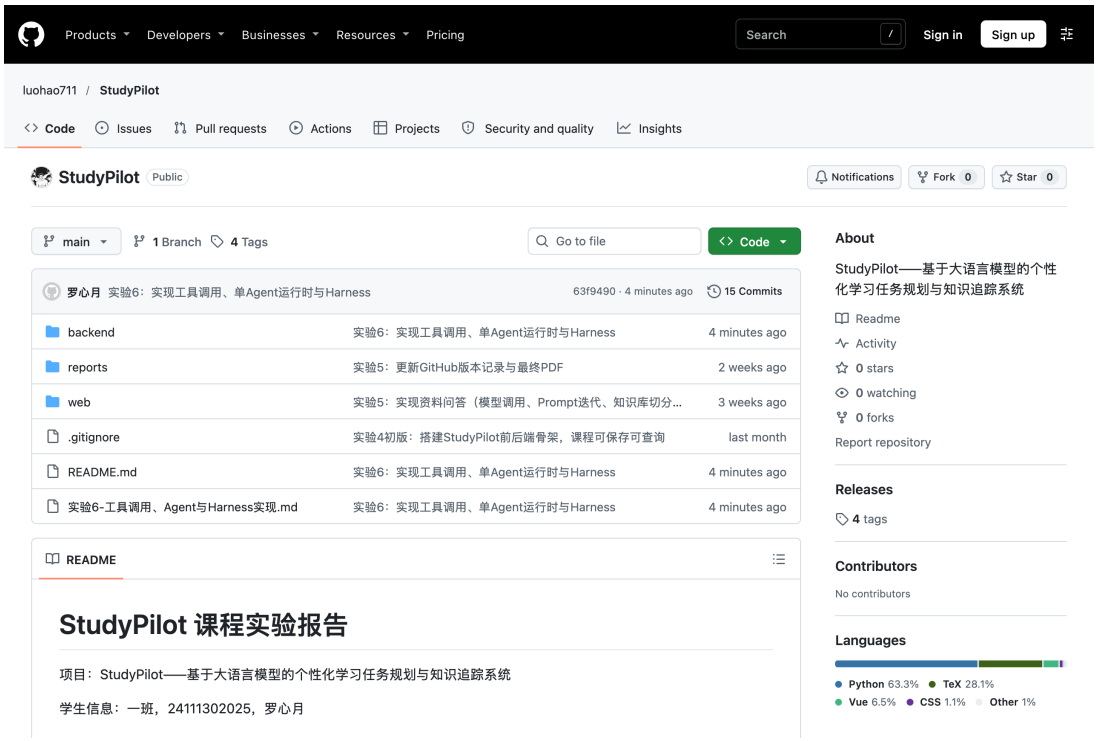


图 6-17: StudyPilot 仓库页面（最新提交为实验 6）

4. 实验小结

本次完成了 StudyPilot 的单 Agent 学习情况分析功能：模型通过 4 个工具查询课程、任务和统计结果，完成率低于 60% 时为最薄弱知识点创建 30 分钟复习任务，写操作必须由用户确认；外围 Harness 负责上下文、工具分发、参数与权限检查、步数/超时/重试限制、结果核验和日志。

过程中主要做了四件事。第一是确定任务边界：把范围缩小到“软件工程最近 14 天”，用 12 条样例任务和 4 个知识点构造出完成率 41.7%、薄弱点“软件测试/数据库设计”的可核对场景，成功标准全部写成程序可检查的数字。第二是工具层：4 个工具都用 Pydantic 定义参数，只允许注册表内的工具，查询只访问当前用户的数据且 SQL 预先写死；接入 Agent 前直接测试了正确参数、越界参数、类型错误、课程不存在、未知工具、写操作参数和访问范围错误，错误调用没有写入数据。第三是循环和限制：一步定义为一次模型请求，最多 8 步、总时限 90 秒、模型单次 30 秒且重试 1 次、只读工具 3 秒且重试 2 次、写工具 5 秒不自动重试、确认等待 20 秒，这些都由程序执行；工具调用和结果按 `tool_call_id` 加入上下文，日志里能看到上下文从 2 条增长到 8 条。第四是结果检查和失败处理：最终回答由程序独立查数据库逐项核验，真实运行 4 步完成、创建任务 #486；拒绝场景不写入；参数错误、工具异常、步数/时间上限和核验失败都能给出明确状态和原因，最终回答不通过时最多修正 2 次。

测试共 43 个用例全部通过，其中 23 个是本实验新增的工具、循环、限制、确认、核验和日志用例；真实模型验证了正常完成、确认同意和确认拒绝三条路径，其他异常路径用脚本模型稳定复现。当前局限是任务限定单课程、命令行确认、日志无轮转、超时只能停止等待而不能真正终止后台线程、最终回答格式较严格；下一步把 Agent 接入页面和接口，增加任务完成/计划调整等需要确认的工具，补充运行列表页和回归测试集。